

[Maks Burchard - 240894 - Final Report]

[Block Y1B-2024-25]

Introduction

Sometimes, some things are unclear. Especially in the world of healthcare, where patients may not be willing or able to disclose information. The goal of this report is to predict the age of the patient based on available data, so hospitals and medical clinics have a better view on what may be best for a patient.

The dataset comes from the University of California Irvine's Machine Learning Repository and encompasses 50 features, each having 101.766 rows. The data was collected between 1998 and 2008 from 130 hospitals and integrated delivery networks in the United States of America, from patients who have had diagnosed diabetes and stayed up to 14 days in a hospital. Features include IDs for patients and encounters, whether certain drugs were prescribed or changed, personal information and medical history of the patients.

The three machine learning objectives I did in the Creative Brief are predicting, classifying and clustering. I predicted the age of a patient with the use of linear regression, decision tree regression, gradient boosting regression, and XGBoost regression. After that, I classified the data once again into age, using models such as Logistic Regression, Random Forest Classifier, and K-Nearest Neighbour. Finally, I clustered the data using K-Means Clustering, Random Hierarchical Clustering, and DBSCAN. I then evaluated the different models and decided on the best ones for each task.

Problem Statement

The main focus during the Creative Brief was on regression, as this is what fits the best to the task at hand, where the goal is to predict the age based on other features.

1) Exploratory Data Analysis (EDA)

The first step after one has collected data is the Exploratory Data Analysis (EDA). The goal of the EDA is to orient yourself on what kind of data you have collected. Questions such as "How much data is there?", "What does each feature mean?", "Is there something special happening", etc. are imperative to this step. A bad EDA can lead to assumptions being made that influence models later on. This was the goal of the first chapter of the Creative Brief.

Task 1A: Exploratory Data Analysis with SQL

The first part of the EDA was to use SQL to explore the data. Apart from just learning about the data, I learned how to use SQL. I had a bit of experience with basic EDA from the data in the previous block but this time I had access to more powerful tools, with which I could sketch a clearer picture for myself.

1A.1: General Overview of the Data

The first step in the EDA was to just get to know what you're working with. For this, basic operations need to be executed, one of which is to get the total number of records, so you know just how much data you have. As well as that, basic operations regarding the distribution of the data in certain columns can be done as well. Two examples which I did here were to check the distribution of different admission types and to explore the top discharge dispositions. From this I knew some basic information about the data, such as the admission type being more than 50% of one type and that the top discharge disposition id was 1, which meant that the patient was discharged to home.

1A.2: Identify Missing or Anomalous Data

Missing data is the most type of data that isn't helpful. Therefore, great care must be put in the way one deals with missing data in their dataset. Whether it is removed by way of dropping or filled by a certain metric, missing data should always be dealt with. It is therefore of the utmost importance that one familiarizes themselves with the missing data within their dataset during the EDA. This was my goal during this step. I checked for missing and/or unusual data in the "race" and "weight" columns. I noticed that "race" had a small amount of data formatted as "?", most likely meaning that it was missing. "Weight", on the other hand, had a staggering amount of "?", with the number being close to 100k, and as I knew the data was only 101.766 rows, this meant that a very high amount of "weight" data was missing. This would be something that needed to be tackled later on.

1A.3: Understanding Age Distribution

After having explored the dataset for a bit, it was now time to get a clearer and more specific picture of the data. The "age" column could be helpful with this. It would give an indication of what kind of people, children, adults or seniors, were most represented by the dataset or whether the amount was balanced. After checking the distribution, I saw that an overwhelming amount of people were between 50 and 90 years old. This was not too surprising, as seniors are more likely to be sick and have diabetes.

1A.4: Admission Trends by Source and Type

Another way to get information is to group two columns with each other. Here I did that with hospital admission, admission sources and admission types. The data this produced was not very readable as the same number came up multiple times in each column, but it did give a basic overview of correlations between the two columns.

1A.5: Hospital Stay and Readmission Patterns

Finding the average time someone spent against what state they were admitted in should give a clear correlation, as someone admitted in a worse state should on average spend a longer time being treated than someone who came in a lighter state. I decided to check this and indeed, such a correlation did seem to be happening. However, correlation $\neq$ causation and therefore, domain knowledge would have to be implored to confirm this suspicion. I also decided to check the readmission rates by admission type to see if any pattern was present there, but nothing seemed to jump out at me, although I did note some admission types having higher readmission rates.

1A.6 Comparing Admission Types and Outcomes

The final step I did was to continue with this correlation between two columns. This time, I compared discharge disposition to admission types and readmission rates, to see if there were any relationships between the columns. I noted that some discharge dispositions seemed to be skewed towards certain admission types and readmission rates, which could potentially mean that there is a cause here, but as I stated before, correlation $\neq$ causation.

Task 1B: Exploratory Data Analysis with Python

The second part of the EDA was to use Python and its packages to explore the data. This would set more of a foundation for me, as the rest of the Creative Brief was in Python as well. I already had some knowledge on the data from my EDA with SQL, but this EDA would also give me information that was more suited to be used in later parts of the Creative Brief.

1B.1: Load and Analyse the Dataset Shape

The first logical step in the EDA is to load the data, so that's what I did, at first using Pandas. I also checked the basic information for the data, such as shape and column names, and checked the first five rows to see if the data had loaded correctly. Everything looked fine to me, so I continued.

1B.2: Load and Explore a Dataset Using NumPy

After using Pandas to do the EDA, I used NumPy to almost achieve the same results. NumPy had a disadvantage to Pandas in that it only loaded the data in a 1D array instead of the 2D dataframe, so I decided to use Pandas to load my data going forwards.

1B.3: Analysing Data Types

The next step in the EDA was to check how Python saw the data. I had loaded the data and shown the first few rows and from there I could make a good guess on how Python saw it, but it never hurts to make sure. I printed the data types to see if me and Python agreed on them, and while we did for the most part, two columns that stood out were "age" and "weight", with them both being "object" types, i.e. Python saw them as categorical values, while being more appropriate for numeric values. I knew why "age" was seen this way, as when I printed the data out in steps 1B.1 and 1B.2 I saw it being formatted in a way that Python saw it as categorical data. But "weight" did stand out to me, as all 5 rows I printed out had missing values in the "weight" column. I made a note of this to come back later. I also checked the unique values to see if I could gain new insights into that, and I noted three things: First, I now knew that the "weight" column was formatted in a similar way to the "age" column, and that that was the reason why Python was seeing it as categorical data. The other two things I noticed were that both the "examide" and "citoglipton" columns had one value, and would therefore be useless for any task.

2) Data Processing

Defining why missing values are detected, and why importing metadata is important, including justifications for using each visualization

Task 2.1: Initial Cleaning and Pre-processing and Visualisation

After the EDA, the next few steps are regarding preparing the data to be used by the Machine Learning Models. They are sensitive to incorrect and missing data, therefore that must be taken care of.

2.1.1: Load and Check for Incorrect Types

I already did this in 1B.3, as during my analysis of the data during that step I noticed the “age” and “weight” columns being seen as categorical instead of numeric data. However, this was due to the way both columns were formatted, so to fix this I would need to replace the values, which would be more appropriate to be done during the encoding process, along with the other categorical columns.

2.1.2: Identify and Remove Duplicate Rows

Duplicate Rows can skew the data towards the values that are duplicated and should therefore be removed. Luckily, this dataset didn’t seem to have any duplicate rows, but I added in the code anyway as it’s better to be safe than sorry.

2.1.3: EDA with Visualisation

Visualizing data can be a powerful way to explore data. If done correctly, you can easily see patterns, if something is lopsided, outliers and much more. I have to justify the use of each visualisation in my Creative Brief.

The age histogram immediately and clearly shows the distribution of the age of patient. This shows that the dataset is biased towards older patients.

The gender histogram shows that the female/male ratio is pretty even, with females having the slight edge. It also shows that a third value, Unknown/Invalid, is present in the column in small numbers.

The time in hospital boxplot shows that the average time patients spent in the hospital is about 42, and that most of them spent between 30 and 60 days in the hospital. There were some notable outliers all the way up to about 130 days.

The insulin histogram shows that the data in the column is pretty spread out between the 4 values, although “No” appears most often.

The metformin histogram shows that the column is mainly filled with “No” with “Steady” appearing a steady amount while “Up” and “Down” appear very scarcely.

The readmitted pie chart shows that the majority of patients don’t get readmitted, and between those who do, about 2/3rds get readmitted within 30 days.

The race histogram shows that the main race of patients in the dataset is Caucasian, with African American in 2nd and having more data than the remaining 4 values combined.

The stacked race bar graph shows that the way Caucasians get admitted is more diverse than African Americans. The other four values are too small for me to be able to draw any conclusions from them.

The medical specialty histogram, while very cluttered, shows that almost 50,000 values, close to half the dataset, are “Pediatrics-Endocrinology”, and the rest of the data is very spread out between the remaining values.

The stacked medical specialty readmission bar graph shows the ratio between readmitted and non-readmitted patients is similar, no matter what specialty, with only “Emergency/Trauma” having a noticeably higher readmission rate.

The number of diagnoses histogram shows that an overwhelming number of patients get diagnosed 9 times. The rest of the data is concentrated below that number, while only a small number get diagnosed more than 9 times.

The final chart is a simple histogram showing the distribution of values in the “diabetesMed” column, showing that the majority of patients get prescribed diabetes medications.

Task 2.2: Identify Metadata and Missing values from the Dataset

Metadata is data that gives information about other data. Missing values are data that have no information. In some special cases, this can be helpful, but not in machine learning. Both types of data are important to know about, but each for a different reason. Metadata can inform you about the data that you have while missing data needs to be dealt with.

2.2.1: Identify Metadata from the Dataset

As mentioned, metadata can inform you about the data you are working with, similar to the EDA. It's simple to understand and easy to work with, and therefore tools have been created to easily produce the metadata.

2.2.2: Identify Missing Values

As I stated multiple times before, missing values are no good for machine learning. The goal of this step is to identify which columns have missing data. Luckily, similar to metadata, due to the importance of dealing with missing values, tools have been created specifically for this step. Calculating the % of missing values can help you decide whether to drop the entire column or to proceed with alternate methods, such as filling the data or dropping the rows. A simple search for the amount of nan values reveals two columns with nan values: “max_glu_serum” and “A1Cresult”. Knowing the shape of the data from previous steps and seeing the amount of nan values here made it immediately clear that both columns had a large percentage of missing values, and after calculations, this turned out to be correct, as the columns had 95% and 83% of their values missing respectively.

However, the analysis I done here had a problem: it only searched for nan values, and missing values can be formatted in a few ways. One of these ways was with a question mark, which I knew multiple columns in this dataset were using due to checking the unique values of each column. So, I did a 2nd analysis, where I replaced all ? values with nan before checking and this revealed multiple more columns as having missing data as well. The big one was “weight”, consisting of 97% of missing values. The columns “payer_code” and “medical_specialty” also had large amounts of missing values with 40% and 49% respectively. Other columns such as “race”, “diag_1”, “diag_2”, and “diag_3” also had missing values, but in small amounts, as none of those exceeded 2,5%.

Two other unique values which I had to look at were *other* in the “race” column and *unknown/invalid* in the “gender” column. These were ways in which missing values were sometimes formatted and as this dataset already had two ways in which they were, I wanted to make sure I didn’t miss anything. I determined that the *other* value in the “race” column wasn’t missing values as the column already contained the ? value. Gender, on the other hand, I wasn’t too sure what to do with, as in this progressive world, “Unknown” may be a valid way some people feel about their gender and as I wasn’t a part of the LGBTQ+ community, I do not feel comfortable with potentially excluding those people. I knew because of the visualizations in an earlier step that there was not a large amount of these values, so model would not be too majorly impacted, so I decided to leave the values in.

2.2.3: Splitting Data for Analysis

The final step before working on the models is to split the data. This step has to come after the processing steps as both the train and test datasets need to be in the same format, but before any work with models to prevent data leakage.

3) Machine Learning Models

Task 3.1: Implementing a Regression Model

First of the three tasks is regression, where a model predicts the value based on other features.

3.1.1: Loading the data

I had done this already in previous steps, so I re-used the code.

3.1.2: Implement a Linear Regression Model

I decided to load the data in the beginning here for two reasons: first, if I ever restarted the kernel due to either closing the Creative Brief or to force a runaway model to stop, I had to run only this one cell to have the data be the same again. Secondly, if I ran into an error somewhere I had all the code close together and I wouldn’t be forced to scroll up and down the document to debug.

Due to the same reason as stated above and because you had to process the data after you loaded it, I put the pre-processing steps here as well. I decided to drop all the columns with large amounts of missing values and to drop the small number of rows containing missing values in

other columns due to that being categorical data and therefore not able to be filled. After that, I encoded all categorical variables with label and one hot encoders, before scaling the features. As a final step before the model, I split the data into train and test sets.

The model here was a simple linear regression model that I predicted the age of the patient with. I then calculated the mean absolute error (MAE) and mean squared error (MSE) of the model.

3.1.3: Visualise the output for linear regression

Visualising the output of the model can give you insights in how it works. In this scatter plot, on the y-axis are the values the model predicted while on the x-axis are the actual values. The actual values are divided into 10 bins due to the way they were formatted into 10-year categories in the original dataset. It is clear from this plot that the model mainly predicts between 0.4 and 0.9, with only one outlier below.

3.1.4: Implement a Decision Tree Regression for Non-linear Relationships

A 2nd model that can be used for regression is a decision tree regression. I used it here on the same data as the linear regressor to be able to compare the two using their MAE and MSE scores. I did a small bit of manual hyperparameter tuning and this looked to be the lowest I could get the scores.

3.1.5: Implement a Gradient Boosting Regression

A third model that can be used for regression is a gradient boosting regressor, which uses weak learners (mainly decision trees) and updates them to create a better model. I once again used the same data as the previous two models for comparison using the MAE and MSE scores. Just like with the decision tree regressor, I did a small amount of manual hyperparameter tuning on the model.

3.1.6: Implement XGBoost Regression

The fourth and final model I implemented for regression was XGBoost, which aims to improve on gradient boosting. I had a couple of problems with the regressor not accepting continuous values. I looked around the documentation for the regressor, but I could not find a clear way of getting the regressor to work with continuous values, so I had to cut the y train and test sets into 10 bins before training the model. This could explain why the MAE and MSE values are so high for this model.

3.1.7: Visualise the output for non-linear regression

As stated before in my justification of the scatter plot of the linear regressor output, visualising the output of the model can give you insights in how it works. The same is true here: You can see that all models have a spike between 0.6 and 0.7, although their distributions are different. From the visuals it can be seen that the decision tree regressor predicts values between 0.2 and 0.9 while the gradient boosting regressor predicts between 0.5 and 0.8. The XGBoost regressor has a major gap between about 5.25 and 5.90, but I cannot figure out what is causing this.

3.1.8: Evaluation and compare the performance of the models

Comparing models is important to figuring out which one is the best. I have done that here by comparing the MAE and MSE scores from each model. The decision tree just slightly beats gradient boosting and linear regression, while XGBoost is far behind. The decision tree and gradient boosting are so close together that it is not completely impossible to rule out chance, as the random state I chose could have influenced this battle.

3.2: Implementing Classification Baseline

The second of the three tasks is classification, where the goal is to classify a value to a certain label based off of the other features.

3.2.1: Loading the data

I had done this already in previous steps, so I re-used the code.

3.2.2: Prepare Data for Classification

I once again loaded, pre-processed, scaled and split the data all in the same cell here. I justified this before in part 3.1.2.

3.2.3: Implement Logistic Regression Model

The first model for classification was the logistic regression model. I trained it and then let it classify the data. All regression models were going to be evaluated using the accuracy, prediction, recall and f1 scores.

3.2.4: Implement a Random Forest Classifier as a Non-linear Baseline

The second model for classification was the random forest classifier. Similar to the first model, I trained it and then let it classify the data before evaluating it using the 4 scores mentioned before.

3.2.5: Implement a K-Nearest Neighbours (KNN) Classifier

The third and final classification model was the K-Nearest Neighbours classifier. I had a slight bit of experience previously with the k-centre algorithm with regards to the shortest path, so I knew the basics of the model. As is immediately visible from the code, I did hyperparameter tuning on the algorithm for the `n_neighbours` parameter. I calculated the four evaluation metrics for each value of the parameter between 2 and 20 and then visualised it in a graph to easily tell which one had the highest metrics. From said visual, it can be concluded that the optimal value for the parameter is 17, as two of the four metrics are highest there and it comes right before a drop in all values (note: the “accuracy” and “recall” metrics line up perfectly, so therefore it seems like there are only three values being plotted.)

3.2.6: Evaluation and Model Comparison

Once again, evaluation and comparison is important to conclude which model is best for the task. I used the four evaluations I calculated and compared them to decide that the best model seemed to be the random forest classifier, as it had the highest score in all four metrics.

Task 3.3: Implementing a Clustering Model

The third and final machine learning task done in this creative brief was clustering.

3.3.1: Load and Preprocess the Data (Already done in previous tasks)

I had already done these steps in previous steps, so I re-used code here. My justification for this remains the same as previous times.

3.3.2: Implement K-Means Clustering

The first way I did clustering was using K-Means clustering. I used hyperparameter optimization for the number of clusters by running all values from 2 to 10 and then getting the inertia and silhouette score for each. I plotted the inertia against the number of clusters to choose the optimal value based on the elbow method, but the graph did not have an obvious bend and more of a gradual descend. I chose the value 5 as it was towards the end of the bend and had the last high silhouette score before a large drop.

3.3.3: Apply PCA for Dimensionality Reduction

The next clustering model, random hierarchical clustering, can be computationally intensive, so therefore it is imperative that the dimensionality be reduced to prevent the model from crashing the computer. I reduced the data to 2 and 3 dimensions to visualise it in a scatterplot, which I created here. The plots enable the ability to quickly look for patterns and/or outliers in the data. Due to the density of this dataset, you cannot easily see patterns, if there even are any. A couple of outliers do become visible.

3.3.4: Implement Random Hierarchical Clustering

After reducing the dimensionality, it was time to implement the model. I had some troubles with the model taking an abnormally long amount of time to compute, and therefore I changed some of the default parameters to try and get the model to take less time, which I managed to not quite achieve, so I decided to sample the dataset to train the model on a smaller amount of data with the hope of it taking less time, which worked. I once again used the silhouette score for evaluation of the model.

3.3.5: Implement DBSCAN for Density-Based Clustering

The final clustering model was DBSCAN, which I implemented without any major obstacles. I once again did a small amount of manual hyperparameter tuning and calculated the silhouette score of the model.

3.3.6: Evaluation and Model Comparison

As stated before, comparing models using a shared evaluation method is an easy way to quickly identify the best model for a task. For clustering, the evaluation method used was silhouette score, and the best model was random hierarchical clustering, as it had the silhouette score closest to 1.

3.4: Feature Engineering and Feature Selection

Reducing the number of features can both improve model performance by letting it focus on relevant features and reduce computational power and time, due to the models using less data to get results.

3.4.1: Feature Engineering

Feature engineering is the art of either changing or creating features in the dataset. An example of changing features is what I did here, namely replacing the semi-numerical values of the “age” column into a fully categorical one. An example of creating features is if a dataset contained the columns of “wage per hour” and “hours worked”, you could calculate the yearly salaries by multiplying the two columns. The newly created columns could then be used for a variety of purposes.

3.4.2: Generate Statistical Features

This step involved calculating statistics of columns in the dataset. While the step required me to only calculate the features of certain columns, I decided to do a bit more and calculate the mean, median, maximum, minimum and standard deviation for every numerical column in the dataset.

3.4.3: Feature Transformation

Normalisation and standardization are important for a few reasons. First, some models, especially in clustering, work based on the distance between certain data points. It is then important to make sure all features are on a similar plane. Secondly, other models which don't work on distance can overfit to a column with large values and be less efficient in predicting or classifying values.

3.4.4: Feature Selection

Feature selection is the step where important features are identified and noted. I did the identification through a combination of the importance identified by the random forest classifier (rfc) and XGBoost regressor. I divided the importance as given by the rfc by the importance given by XGBoost. This created a bit of a problem as XGBoost can remove features from its importance list while rfc cannot. And since dividing by 0 is not possible, this would cause an error. Instead of simply swapping the numerator and denominator, I decided to challenge myself to try and solve this problem by dropping the columns missing in the XGBoost importance list in the rfc list. While I did struggle a bit with this, eventually I managed to get it working. I then sorted the columns from highest to lowest ratio and based off of that I decided to keep the first 13 features and the “age” column.

3.4.5 Model Evaluation After Feature Selection

After selecting the features, I re-trained the linear regression model on the new dataset. I once again loaded, pre-processed, scaled and split the dataset within the cell. This was more important now as I was dropping columns other model could have used in previous parts. I once again calculate the MAE and MSE of the model, which showed an improvement on the previous version.

Task 3.5: Hyperparameter Tuning

Hyperparameter tuning is the process of checking different parameters to see which one yields the best results. I had already done a bit of this in previous steps, both manually and semi-automatically, but here it would be done within one function.

3.5.1: GridSearchCV for Systematic Hyperparameter Tuning

GridSearchCV is a function that takes a grid of parameters and runs through all possible combinations. That is what I would use to tune the hyperparameters here. The first step while using GridSearchCV is to create the parameter grid. I decided on tuning the hyperparameters of three models: decision tree regressor for regression, logistic regressor for classification and K-means for clustering. I created the grid for each of the models here. For the logistic regressor, I decided to include the 'l1_ratio' parameter, knowing it would make some of the folds fail as it was only required for the 'elasticnet' penalty parameter.

3.5.2: Apply GridSearchCV with Cross-Validation

After creating the grid, I had to now use GridSearchCV with the selected models. I split each count of GridSearchCV into its own cell as the process was computationally heavy and this would be helpful for debugging, since it wouldn't require me to wait for the previous functions to finish before it showed an error. This step went well with no problems.

3.5.3: Use Nested Cross-Validation to Validate Stability of Hyperparameters

After running the GridSearchCV it was time to check how well it had performed. For this purpose, I used nested cross-validation. The first two implementations went fine but the final one with K-means I had some trouble with regarding multiple errors that popped up, but I eventually managed to work through all of them.

3.5.4: Evaluate Tuned Models with Multiple Metrics

The final step of hyperparameter tuning was to evaluate the models using multiple metrics. Evaluated the models using the metrics I evaluated them earlier with, except for clustering, which only had 1 metric originally. I calculated the Calinski-Harabasz index for the tuned K-means model.

Task 3.6: Model Selection

The final part of machine learning is to select the best models and save them for potential later use.

3.6.1: Compare and Choose the Best Model

I already compiled the evaluation of all models at the end of each modelling task, so I copied those tables here and made the decision of the best model for each task final: decision tree regressor for regression, random forest classifier for classification, and random hierarchical clustering for clustering.

3.6.2: Finalise and Save the Best Model for Each task

The very final step in this creative brief was to save all three models using joblib. I already trained all models on the full dataset, so all I had to do was use `joblib.dump` to save the models to a file. A problem I encountered with this was the fact that the random forest classifier model would save to a file that is 1.16GB large. This is a problem because GitHub had a file upload limit of 100mb. After compressing the file as much as I could it was still almost 50% over the limit at 145mb. I decided that I could not upload this file to my GitHub repository.

4) Conclusion and Reflection

After all of this work, I have chosen three models which I believe, due to my comparisons and evaluations, to be the best for their respective tasks. These models are:

Regression: decision tree regressor

The decision tree regressor, despite being the very basic and the base for two of the other models, beats the others' MAE and MSE scores both narrowly and by a wide margin. While without context, this may seem a bit weird, analysis of my work and its results reveals a bit of nuance: namely, gradient boosting lost by a small amount and XGBoost had problems which required changing the data a bit for them to be solved. This could potentially mean that I made errors in my work which if done correctly, could have led to a different outcome.

Classification: random forest classifier

The random forest classifier beats the other models' metrics by a similar amount every time. This would make it seem an obvious choice, but some doubt can be found since it only gets 31% of it's predictions correct, which is a very low amount. This could be an area for potential future exploration to see what is causing this, whether it be the dataset or my implementation.

Clustering: random hierarchical clustering

The random hierarchical clustering beats the other models in the most convincing fashion out of all three tasks. It is the clear winner in this category and while no errors seem to be obvious, doubt

that is caused by the results in the other two tasks could be reason for further investigation of my results.

In conclusion, the results of my work here may potentially have some flaws. This can potentially be reason for future work to re-evaluate what I have done here to see whether my results are a fluke or truly correct.

5) References (if there is any)

Diabetes 130-US Hospitals for Years 1999-2008. (2014, February 5). UC Irvine Machine Learning Repository. <https://archive.ics.uci.edu/dataset/296/diabetes+130-us+hospitals+for+years+1999-2008>